AUTONOMOUS WAREHOUSE AMR SIMULATION REPORT

Lane-following pick-and-load robot with swappable suction-arm and conveyor modules, simulated in PyBullet

Simulator	PyBullet (Bullet physics), Python 3.11, OpenCV
Robot	Differential-drive AMR, 0.70 m x 0.62 m chassis, 0.10 m wheel radius
Sensors on the AMR	2 forward RGB-D cameras, 1 centre RGB lane camera, IMU gyro, 2 wheel encoders
Payload modules	Suction arm (6-DOF, eye-in-hand stereo) and conveyor (lift, belt, IR beams, ToF)
Warehouse	22 m x 18 m, 3 rack aisles, 4 dock bays, painted yellow lane network
Mission	Two stock locations (A: four small boxes, B: one large box) to a truck, then home
Result of the recorded run	Mission complete in 865 s of simulated time, 5 of 5 boxes in the truck

1. Objective

The aim of this work was to build a warehouse simulation in which a single mobile robot does a complete pick-and-load job on its own. The robot is not told where the boxes are, how many there are or how big they are. It has to drive to each stock location, look at what is there, decide which tool it needs, go and fetch that tool from its stand, load the goods, take them to the truck, put the tool back and finally return home. Nobody drives it and no route is scripted.

We also wanted the simulation to be honest about sensing. The controller never reads the robot's true position from the simulator. Its position comes from wheel encoders, a gyro, visual odometry and the painted lane, and the obstacle map is built from the depth cameras while the robot drives. The arm only knows where a box is through its own stereo camera, and the conveyor only knows where the box is through its IR beams, range sensor and belt load sensing. This keeps the software close to what would run on a real robot.

2. Simulation Environment

The warehouse is 22 m long and 18 m wide. Along the west wall there are four dock bays: Shipping (where the truck is backed in), Picking (arm-module stand), Loading and Unloading (the robot's home position) and Reception (conveyor-module stand). The east half holds three rack aisles. A yellow lane, 0.6 m wide, is painted on the floor. It runs from each bay to a north-south spine and from the spine into two of the aisles, to the pick table at point A and to shelf B.

Six obstacles were deliberately placed on or next to the lane: a pallet on the home spur, barrels on the spine, a trolley on the spine near the shipping bay, a crate in the middle aisle, a cart in aisle 0 and a bin near the truck. They force the robot to leave the lane and come back, which is where most of the navigation work happens. The obstacles are not in the robot's map at the start. It finds them with its cameras.

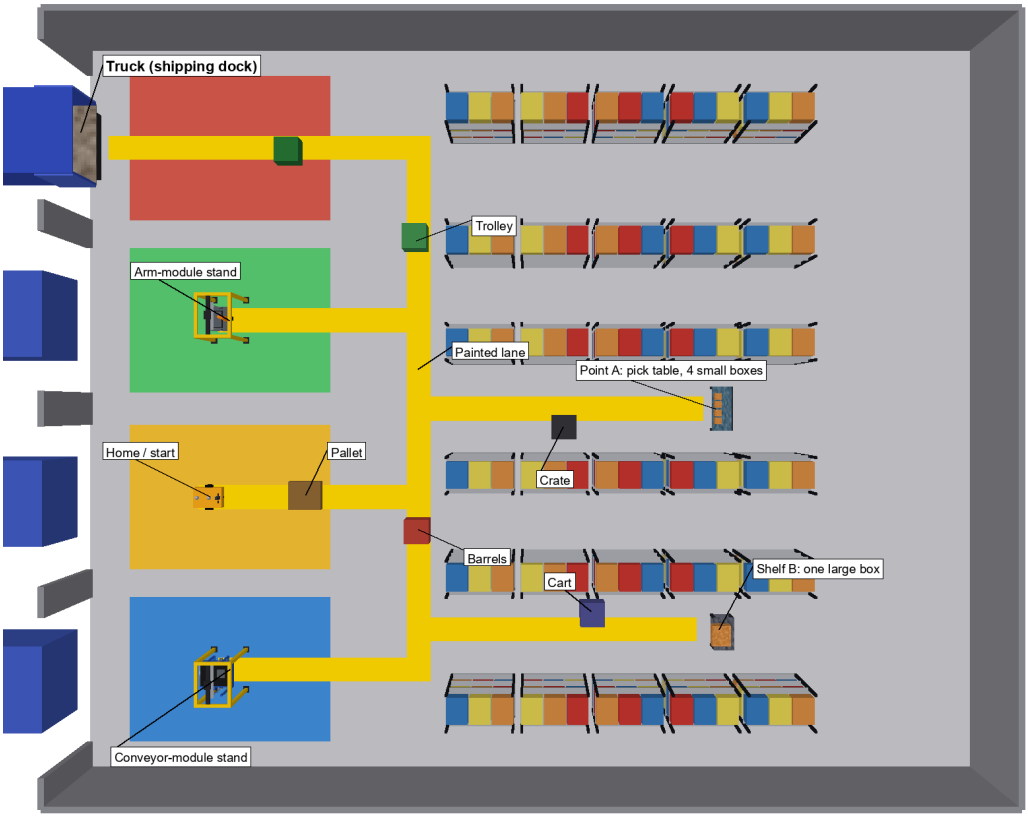


Figure 1. Top view of the simulated warehouse with the lane network, stations, stock locations and obstacles.

Location	Position (x, y) m	What is there
Home	(-8.0, -2.2)	Start and end pose in the Loading and Unloading bay
Arm-module stand	(-7.5, 2.2)	Portal stand holding the suction-arm module, with a hoist
Conveyor-module stand	(-7.5, -6.5)	Portal stand holding the conveyor module, with a hoist
Point A	about (4.6, 0.0)	Pick table with four 0.16 m cartons (1 kg each)
Shelf B	about (4.7, -5.5)	Shelf top at 0.62 m with one 0.45 x 0.50 x 0.32 m carton (3 kg)
Truck	rear sill at x = -10.7	Hollow trailer, deck about 0.40 m above the floor

3. Robot and Modules

The base robot is a two-wheel differential drive with casters. It has a sensor mast at the front carrying three cameras. The two RGB-D cameras (left and right, 0.31 m above the floor) are used for mapping, obstacle avoidance and box detection. The centre RGB camera is pitched down and is used only to see the painted lane. A gyro provides the heading rate and both drive wheels have encoders.

The top deck has two magnet pads. A payload module is lowered onto the deck by the hoist on its stand and locked to these pads, so the same robot can carry either module.

Item	Details used in the simulation
Chassis	0.70 m long, 0.62 m wide over the wheels, deck top at 0.30 m
Drive	Two driven wheels, radius 0.10 m, track 0.56 m; max speed 0.55 m/s in the mission
RGB-D cameras (x2)	Forward facing, 65 deg field of view, used to about 4 m
Lane camera	Centre RGB camera pitched down, lane seen up to about 3.2 m ahead
IMU	Gyro heading rate (used instead of wheel-difference yaw)
Suction-arm module	6-DOF arm: upper arm 0.50 m, forearm 0.45 m, vacuum cup with contact sensing, stereo camera on the flange (75 mm baseline, 320 x 240)
Conveyor module	Belt 0.80 m long and 0.60 m wide, lift 0 to 0.50 m, belt speed 0.25 m/s, two IR through-beams, rear ToF range sensor, belt load sensing

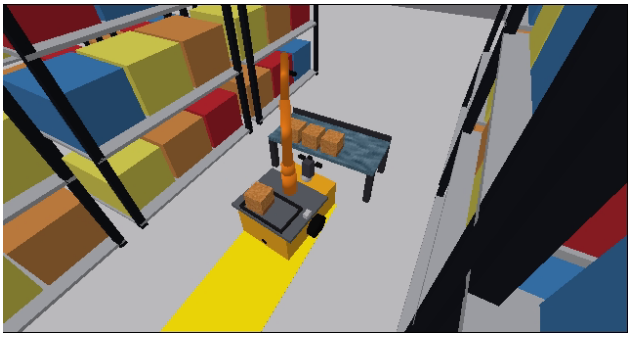


Figure 2. AMR carrying the suction-arm module at the pick table.



Figure 3. AMR carrying the conveyor module with the large box on the belt.

4. Overall Mission Flow

The mission is run by one state machine. The same sequence is repeated for every stock location, and only the middle part changes depending on the module that is needed. The table below lists the steps in the order the robot does them.

Stage	States	What happens
Go and look	EXPLORE_TO_DETECTION, DETECT_BOXES	Drive along the lane to a pose in front of the stock and look at it with the RGB-D camera
Decide	DECIDE_PAYLOAD	Count and size the boxes, choose the module, pin the location on the map
Fetch module	GO_TO_STATION, LOCALIZE_AT_STATION, TURN_FOR_DOCK, REVERSE_DOCK, TRANSFER_MODULE	Drive to the stand, fix position from the stand legs, reverse in, hoist lowers the module, magnets lock
Load (arm)	GO_TO_PICK, APPROACH_PICK, ARM_LOAD	Park at the table edge and pick every box onto the tray
Load (conveyor)	GO_TO_SHELF, MEASURE_SHELF, TURN_AROUND, REVERSE_TO_SHELF, CONVEYOR_LOAD	Measure the shelf, set the belt height, reverse the nose under the box, pull it aboard
Deliver	GO_TO_TRUCK, MEASURE_TRUCK, APPROACH / REVERSE_TO_TRUCK, ARM_UNLOAD or CONVEYOR_UNLOAD	Drive to the truck, measure the deck, unload
Return module	GO_TO_STATION ... TRANSFER_MODULE	Dock again and the hoist takes the module back
Finish	GO_HOME, MISSION_COMPLETE	Drive home after the last job

Every location the robot discovers is pinned on a semantic map as soon as it has been measured: the stock at A and B, each stand, the pick pose, the shelf edge and the truck sill. Only these final positions are stored. Later steps navigate to the pinned points.

5. Box Detection and Choice of Module

When the robot reaches the detection pose in front of a stock location it takes one RGB-D frame. Two things run on it. A YOLO model (yolo11s) is run on the colour image, but because the COCO classes do not include a cardboard carton its output is only logged. The decision is made by a geometric pass. Pixels with the cardboard colour are segmented in HSV, grouped into blobs and lifted to 3D using the aligned depth image and the robot's estimated pose. Only blobs inside a gate straight ahead of the robot are kept, so the coloured goods on the racks on either side are ignored. For each blob we get its width across the view and its top and bottom height, which gives the box size.

The rule for choosing the module is deliberately simple:

Condition	Module	Reason
No box found	None	Nothing to collect
Largest box footprint 0.40 m or less	Suction arm	A suction cup can hold it and the arm can stack it on the tray
Any box wider than 0.40 m	Conveyor	Too big and heavy for the cup; it has to be slid on and off

In the recorded run the robot saw four boxes of 0.15 to 0.16 m at A and chose the suction arm. At B it saw one box 0.47 m wide and 0.30 m tall and chose the conveyor.

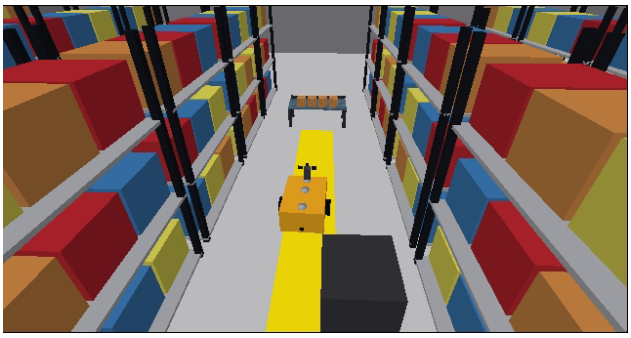


Figure 4. Robot at the detection pose in front of the pick table (point A).

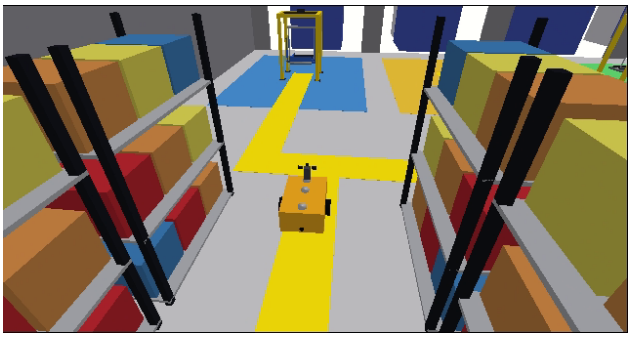


Figure 5. Robot heading for the conveyor stand after deciding on the conveyor for shelf B.

6. Localisation

All mapping and planning depend on the robot knowing where it is, so this part had to be dependable over a 15 minute mission. The pose estimate (x, y, heading) is built from several sources that each cover a weakness of the others.

6.1 Wheel odometry with gyro heading

Distance travelled comes from the two wheel encoders. The heading change comes from the gyro, because the yaw worked out from the difference of the two wheels is spoiled by tyre and caster scrub, and that error changes with the load on the robot. The pose is integrated with the midpoint heading:

$$d = (d_L + d_R) / 2, \quad \theta_{mid} = \theta + \Delta\theta / 2$$
$$x \leftarrow x + d \cos \theta_{mid}, \quad y \leftarrow y + d \sin \theta_{mid}, \quad \theta \leftarrow \theta + \Delta\theta$$

6.2 Visual odometry

ORB features (up to 1000 per frame) are matched between consecutive frames of the left RGB-D camera with a ratio test of 0.75. The matched points from the previous frame are back-projected to 3D with the depth image and the motion is solved with PnP inside RANSAC. When at least 25 inliers support the result and it agrees with the wheels within a gate, it is blended in: 30 % of the heading change and 20 % of the forward distance come from vision.

6.3 Lane-aided correction

The lane is the most useful landmark in this warehouse because it is always in front of the robot. The lane mask from the centre camera is projected onto the floor with a per-pixel look-up table built from the camera mount. The strip is cut into 10 cm bands, a centre point is taken from each band, and a straight line is fitted through them. Since the lanes run parallel to the walls, the direction of a straight strip must be a multiple of 90 degrees, and the difference corrects the heading (gain 0.12, accepted only below 5 degrees). Because the lane layout is known, the sideways offset of the strip also corrects the robot's cross-lane position (gain 0.15, only within 0.5 m and only when the matching lane is unambiguous). The sideways correction is used only while travelling, not during docking or loading, where the robot relies on its own close-range sensing.

6.4 Landmark fixes

At each module stand the robot measures the stand legs with its depth camera before it reverses in, and resets its position from them. The pick table edge, the shelf edge and the truck sill are measured the same way. In the recorded run the position error stayed in the range of a few centimetres to about 30 cm while driving, and the robot finished 1.9 cm from where it believed it was.

7. Occupancy Grid Mapping

The robot starts with an empty obstacle map and builds it while it drives. The map is a log-odds occupancy grid of 220 x 180 cells at 0.10 m resolution, which covers the whole 22 m x 18 m floor. Each cell stores the log of the odds that it is occupied:

$$l = \log(p / (1 - p)), \quad p = 1 / (1 + e^{-l})$$

For every RGB-D frame, the depth image is turned into 3D points using the estimated pose. A ray is traced from the camera to each point. Cells along the ray before the end point get a free update, and the end cell gets an occupied update if the point lies in the obstacle height band (0.10 m to 1.45 m). A floor point counts as free. The whole frame is treated as one measurement, so a cell is updated at most once per frame, and a cell needs at least two end points in the same frame before it receives an occupied vote. This stops single noisy pixels from creating false obstacles.

Parameter	Value
Occupied update	+0.85
Free update	-0.35
Clamp limits	-2.0 to +3.5 (so old evidence can still be overturned)
Free if	$p < 0.35$
Occupied if	$p > 0.65$
Otherwise	unknown

The map has a second layer for the lane. The lane mask from the centre camera is projected onto the floor every frame and voted into the grid: a cell seen as paint more often than as bare floor becomes lane, bare floor becomes not-lane. In the final setup the painted lane layout is also loaded at the start as known map data, because floor paint is fixed infrastructure; the camera votes keep it up to date. Obstacles, boxes, stations and the truck are never preloaded.

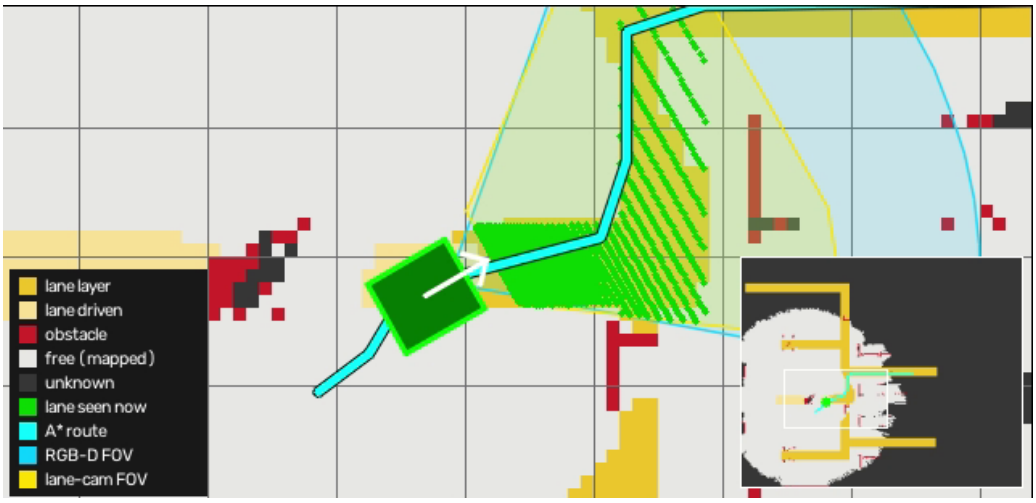


Figure 6. Live occupancy map early in the run. White is free, red is occupied, dark grey is unknown, yellow is the lane layer. The green dots are the lane pixels the camera sees in this frame, the shaded wedges are the camera fields of view and the cyan line is the A* route.

8. Global Path Planning with A*

Routes are planned with A* on the occupancy grid. The grid is 8-connected, diagonal moves cost √2, diagonal moves that would cut a blocked corner are not allowed, and the heuristic is the octile distance to the goal. What makes the planner suit this job is the cost of each cell, which is built so that the lane is always the cheapest place to drive.

8.1 Hard limits

A distance transform gives every cell its clearance to the nearest occupied cell. A cell is blocked if its clearance is less than the robot half-width plus a margin (0.37 m normally, 0.48 m when the wider conveyor module is aboard). Unknown cells right next to an obstacle are also blocked so the robot never squeezes into space it has not seen.

8.2 Cell cost

cost = 1 × (1 + 0.8 · clearance shortfall)

× 1.8 if the cell is unknown

× 60 if the cell is off the lane

× (1 + edge factor) on the lane, so the middle of the paint is preferred

+ 60 · squeeze factor within 0.30 m of the hard limit

The factor of 60 means one metre of floor off the lane costs as much as 60 m on the lane, so A* only leaves the paint where an obstacle blocks it or where the goal itself is off the lane. The squeeze term stops routes from shaving past an obstacle with a few centimetres to spare.

8.3 Lane corridor

A pure cost rule was not enough. When two obstacles are close together, one long diagonal across open floor can cost about the same as two short detours, and the robot cut corners. So the search is now limited to a corridor 0.8 m either side of the painted lane, plus a small area around the start and the goal for reaching off-lane work poses. Near the start the robot may only use cells that are no further from the lane than it already is, so repeated replanning cannot let it drift away. If the corridor is closed the planner tries a 1.4 m corridor, and only if that fails too does it search the whole floor.

8.4 Smoothing

The raw A* path is a chain of cells. It is shortened by line-of-sight shortcuts, but a shortcut is only accepted if its cost is within 3 % of the cells it replaces, and a shortcut between two lane cells must stay on the lane. This keeps the lane corners intact.

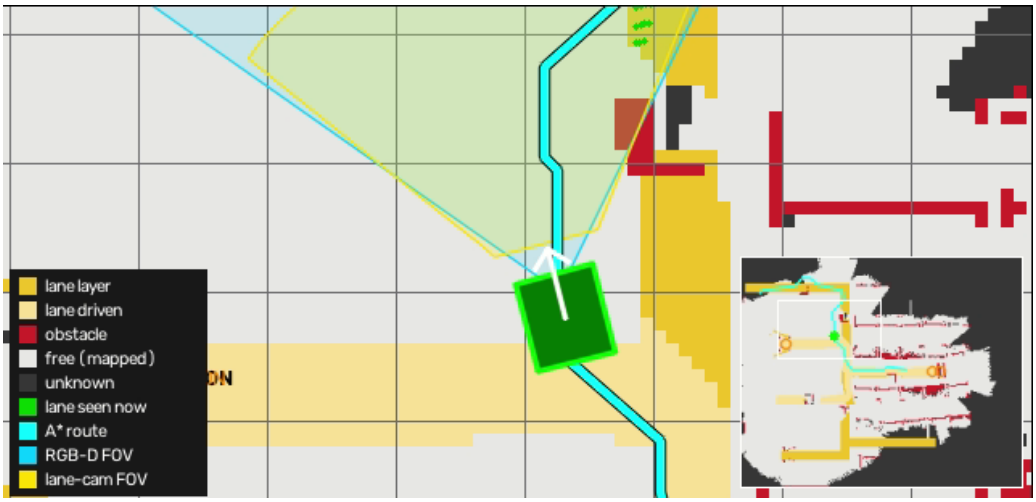


Figure 7. A* route (cyan) on the live map, bending off the spine to pass the trolley (red cells) and returning to the lane.

9. Navigation Priority Order

Following a route sounds easy until several things want to steer the robot at once: the route, an obstacle that the map has not caught yet, the lane camera, a recovery manoeuvre. An early version added them together, and the result was a robot that was pulled back toward the obstacle it was trying to avoid. The navigator therefore works through a fixed priority list every control tick (20 Hz), and the first rule that

applies decides the wheel command.

Priority	Rule	Action
1	Terminal	Arrived or failed: hold still
2	Goal check	Inside tolerance: turn to the requested heading, then report arrived
3	Emergency	Something inside the bumper zone straight ahead: stop and start recovery. Nothing below can override this
4	Recovery	Stop, then back off 0.3 m (or creep forward if the rear is blocked), then replan
5	Route validity	No route, route blocked on the map, robot off the route, or periodic refresh: replan with A*
6	Large heading error	More than 40 deg off the route: turn on the spot. If the turn makes no progress for 4 s the robot is wedged, so recover
7	Local avoidance	The corridor toward the route is blocked by something not yet mapped: steer through the best open gap (VFH)
8	Route following	Pure pursuit on a point 0.6 m ahead on the route
9	Lane following	Lowest priority: while the route runs down the middle of the lane, steer by the paint itself

9.1 Local obstacle avoidance (VFH)

Points from both depth cameras between 0.10 m and 1.45 m high are projected into the robot frame. For each candidate heading from -36 to +36 degrees in 3 degree steps, the planner checks how far the robot could drive straight with its real width (0.38 m half clearance) before touching a point, up to 2.4 m. Avoidance only starts when the heading the robot actually wants is blocked closer than 1.2 m; an obstacle elsewhere in view is not a reason to leave the route. Among the open gaps the score favours small deviation from the route, deep and wide gaps, and short rollouts that stay on the lane, and it penalises switching sides so the robot does not wobble left and right.

9.2 Lane following

When the route runs along the middle of the lane for the next 1.5 m and the camera has a good line fit, the steering comes from the paint: the angle and offset of the fitted centreline. This is what keeps the robot visibly on the yellow line. It is switched off when the route moves to the edge of the lane to pass an obstacle, because the line follower would otherwise pull the robot back to the centre and into that obstacle. That exact problem showed up at the crate in the middle aisle and was fixed this way.

10. Docking and Module Exchange

Each module sits in a portal stand with a hoist. To pick one up, the robot drives to a pose 1.8 m in front of the stand, measures the stand legs with its depth camera to correct its position, turns its back to the stand and reverses in along the dock axis. The hoist then lowers the module onto the deck. PyBullet has no magnetic force, so the electromagnets are modelled as fixed constraints between the module pads and the deck pads. Both pads always lock together, never only one, because a single-point lock lets the module rock. Returning a module is the same manoeuvre in reverse.

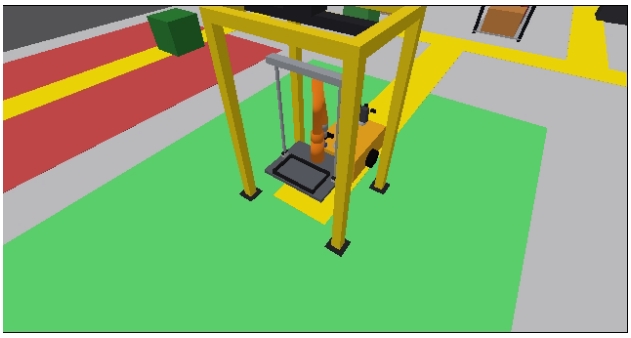


Figure 8. Reversing into the arm-module stand.

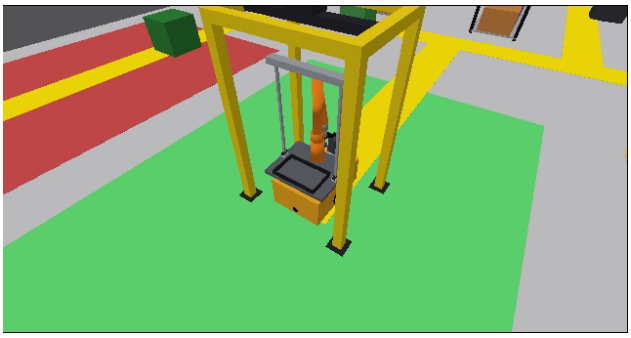


Figure 9. The hoist lowering the arm module onto the robot.

11. Suction Arm: Inverse Kinematics and Stereo Vision

11.1 Kinematic chain

The arm has six revolute joints: shoulder pan (Z), shoulder lift (Y), elbow (Y), wrist 1 (Y), wrist 2 (Z) and wrist 3 (X). Forward kinematics multiplies the 4 x 4 transforms of each joint in turn. The upper arm is L1 = 0.50 m and the forearm L2 = 0.45 m.

11.2 Closed-form inverse kinematics

For picking, the cup always points straight down, which makes a closed-form solution possible. From the required cup position we step up to the wrist centre, then:

$q_1 = \mathrm{atan2}(y_w, x_w)$ (pan toward the target)

$r = \sqrt{(x_w^2 + y_w^2)}, \quad z = z_w - z_{shoulder}$

$D = (r^2 + z^2 - L_1^2 - L_2^2) / (2 L_1 L_2), \quad q_3 = \mathrm{acos}(D)$ (elbow)

$q_2 = -[\ \mathrm{atan2}(z, r) + \mathrm{atan2}(L_2 \sin q_3, L_1 + L_2 \cos q_3)\ ]$ (shoulder, elbow-up)

$q_4 = \pi/2 - (q_2 + q_3)$ (keeps the tool vertical)

$q_5 = 0, \quad q_6 = q_1 + \pi/2 - \psi$ (camera / cup roll)

If |D| is close to 1 the point is out of reach, and any solution outside the joint limits is rejected. The solver was checked against the link poses that PyBullet itself reports.

11.3 Stereo camera on the arm

The arm carries its own stereo pair on the flange (75 mm baseline). The left and right images give a disparity map with OpenCV StereoSGBM, and depth follows from

$$Z = f \cdot B / d$$

where f is the focal length in pixels, B the baseline and d the disparity. The 3D points are moved into the module frame using the arm's own joint encoders and forward kinematics, not the simulator's ground truth. Box top faces are found by colour segmentation of the left image together with these points. The camera also measures what the box stands on, so the box height is known even before it is lifted.

11.4 Pick and place sequence

For each box the arm looks at the stock from above, chooses a complete top face, moves closer and measures it again, then descends slowly. The vacuum cup grips only when its lip is actually in contact with the box, which plays the part of a vacuum sensor. The box is lifted and placed on the cargo tray. At the truck the arm first measures the deck height and the sill, then moves the boxes from the tray into the trailer, first along a row just inside the sill and then a second row behind it. If a box has not touched down where expected, the arm keeps creeping down rather than dropping it in mid-air.

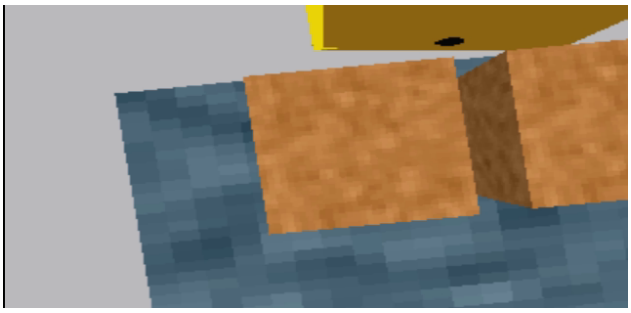


Figure 10. View from the arm's stereo camera above a carton at point A.

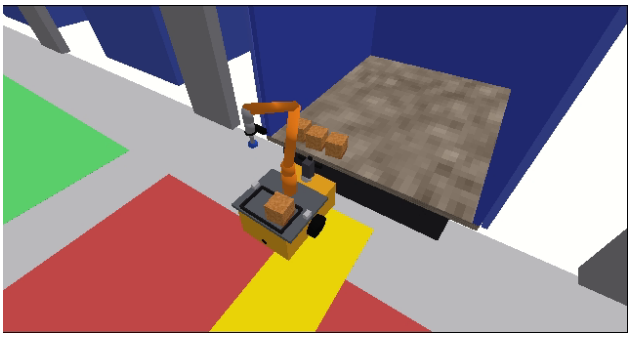


Figure 11. Unloading the cartons into the truck with the suction arm.

12. Conveyor Module

The large box at B cannot be lifted with a suction cup, so it is slid on and off with the conveyor. The module has a belt on a lift, two IR through-beams across the belt (one near the nose, one near the front end stop), a rear-facing time-of-flight sensor under the nose and load sensing on the belt surface.

PyBullet has no moving-surface primitive, so the belt is modelled as traction. Every physics step, each object touching the belt gets a force along the belt that drives its velocity toward the belt speed, limited to what belt friction could deliver (60 N). An object that is not touching the belt is not moved.

$$F = \text{clip}(k \cdot (v_{belt} - v_{box}) \cdot m, \pm 60\ \text{N}), \quad k = 80\ s^{-1}$$

Loading sequence at shelf B:

- The robot stops 0.6 m in front of the shelf and measures the shelf top height, the shelf edge and the box face and centre. If it is more than a few centimetres off the box centreline it re-aligns and measures again.
- The lift sets the belt just below the shelf top. The robot turns round and reverses until the ToF sensor reads about 1.5 cm to the shelf edge, so the nose sits under the overhanging box.
- The lift rises until the belt load sensing feels the box resting on the belt. Only contacts pressing down on the belt count, so brushing the side of the shelf does not trigger it.
- The belt runs inward. The nose beam confirms the box is coming, and the end beam confirms it is fully on. If the nose beam is never broken, the robot lowers the belt, moves out and tries again (up to two retries).
- The belt is lowered to transport height and the robot pulls away.

At the truck the same idea runs in reverse: measure the deck and sill, set the belt slightly above the deck, reverse until the nose is 0.15 m inside the trailer, and run the belt outward until both beams are clear.

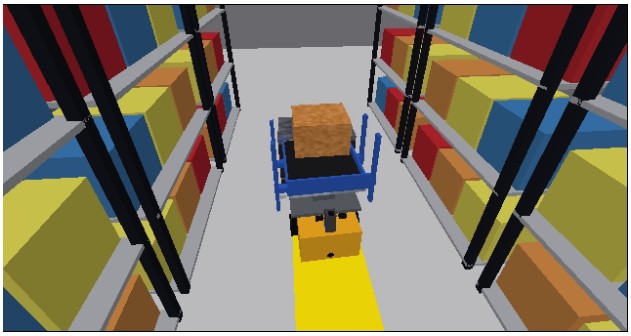


Figure 12. Conveyor nose under the large box at shelf B.

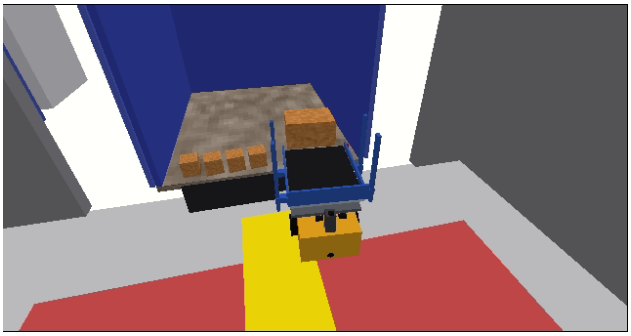


Figure 13. Belt running outward to push the box into the truck.

13. Recording

The mission video is written at 1280 x 720 and 20 frames per second, one frame for every three control ticks, so it plays at three times real speed. The frame has four panels: a chase camera following the robot, the robot's own lane camera (or the arm's stereo camera during picking), the live occupancy map with the route, and a status panel with the state, drive command, estimated pose and its error against ground truth. The camera panels are plain PyBullet renders with nothing drawn over them.

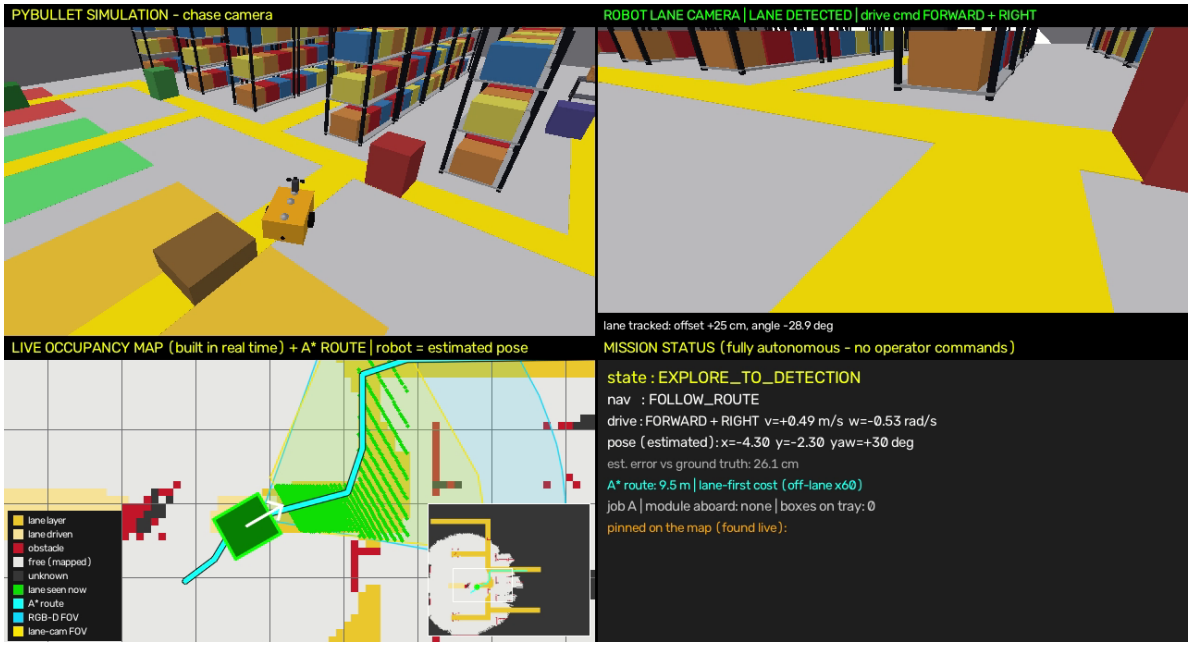


Figure 14. One frame of the mission recording.

14. Results of the Recorded Run

The run below is the final recording (AMR_full_mission_strict_lane.mp4). The robot started with an empty obstacle map and no knowledge of the boxes.

Time (s)	Event
0 to 48	Drives from home along the lane, detours round the pallet, reaches the detection pose at A
48	Sees 4 boxes of about 0.16 m, chooses the suction arm, pins A on the map
72 to 121	Fixes its position at the arm stand, reverses in, module lowered and locked
174 to 260	Parks at the pick table and picks all four cartons onto the tray
264 to 432	Drives to the truck past the crate and trolley, measures the deck, unloads 4 cartons
435 to 500	Returns the arm module to its stand
537	At B sees one box 0.47 m wide, chooses the conveyor
557 to 606	Docks at the conveyor stand and takes the conveyor module
634 to 672	Measures the shelf, re-aligns once, reverses under the box, loads it with the belt
722 to 747	Measures the truck deck, reverses the nose into the trailer, unloads the box
802 to 834	Returns the conveyor module
865	Back home, mission complete

Measure	Value
Mission time (simulated)	865.3 s
Boxes delivered	5 of 5 (4 small by arm, 1 large by conveyor)
Module exchanges	4 (2 pick-ups, 2 returns), all successful
Final position error	1.9 cm
Largest distance from the lane	about 1.0 m on most detours; up to about 1.8 m beside the trolley with the conveyor aboard

15. Problems Found and How They Were Solved

Most of the time on this project went into finding out why the robot did something odd and fixing the cause rather than the symptom. The main ones are listed here because they explain several design choices above.

Problem	Cause	Fix
Robot cut diagonally across open floor instead of following the lane	Off-lane cost alone allowed long shortcuts when two obstacles were close	Lane corridor in A*, widened only if blocked
Robot rubbed the crate while passing it	Line follower pulled it back to the lane centre during the detour	Line following only when the route runs down the middle of the lane
Robot froze turning on the spot next to the pallet	A chassis corner was against the pallet and backing off was blocked	Stalled-turn detection, and recovery can creep forward
Robot slowly drifted beside the lane on long legs	Lane camera corrected heading only	Sideways correction from the known lane layout
Box not detected at the end of the belt	IR ray started inside the box, which ray tests do not report	Beam tested from both rails

16. Limitations

- Everything here is simulated. Belt traction, vacuum grip and magnetic locking are modelled with forces and constraints, not with the real physics of those devices.
- Camera images come from PyBullet's renderer, which is cleaner than a real camera. Lighting changes, motion blur and worn paint are not modelled.
- The box decision uses one size threshold. A real system would also need weight, fragility and stacking rules.
- With the wider conveyor aboard, the gap beside the trolley is tight and the robot takes a wider detour there than anywhere else. Moving that obstacle or narrowing the module would fix it.

17. Conclusion

The simulation shows one mobile robot completing a full pick-and-load job without any operator input. It finds the stock, works out from the box sizes whether it needs the suction arm or the conveyor, fetches that module, loads, delivers to the truck, returns the module and goes home. The obstacle map is built as it drives, routes come from a lane-first A* planner, and a fixed priority order decides between route following, obstacle avoidance, recovery and lane following. The arm uses closed-form inverse kinematics with its own stereo camera, and the conveyor uses only its IR beams, range sensor and belt load sensing.

The next step would be to move this software onto hardware. The structure was kept close to what a real robot needs for that reason: no ground truth anywhere in the control loop, and every decision based on a sensor that exists on the robot.